

▼ Repaso

▼ Scripts

- ☒ Ejercicio 1
- ☒ Ejercicio 2
- ☒ Ejercicio 3

▼ Procedimientos

- ☒ Ejercicio 4
- ☒ Ejercicio 5
- ☒ Ejercicio 6
- ☒ Ejercicio 7
- ☒ Ejercicio 8

▼ Actualizar peso de un jugador

- ☒ Ejercicio 9
- ☒ Ejercicio 10
- ☒ Ejercicio 11

▼ Funciones

▼ Funciones integradas en el SGBD

- ☒ Ejercicio 12
- ☒ Ejercicio 13
- ☒ Ejercicio 14
- ☒ Ejercicio 15
- ☒ Ejercicio 16
- Funciones propias

Repaso

Scripts



Resumen scripts

Definir variables de usuario

```
SET @cod=105, @cad='Hola';
```

Mostrar valores de variables

```
SELECT @cod, @cad;
```

Calcular y asignar valor en la misma instrucción

```
SELECT @numclientes:= COUNT(*) FROM CLIENT;
```

Almacenar datos (sin mostrar)

```
SELECT COUNT(*) INTO @numreg FROM CLIENT;
```

Consultar todas las variables creadas por el usuario (depende de la versión)

```
SELECT * FROM information_schema.USER_VARIABLES;
```

Recuerda

Tipo de variable	Ejemplo	Dónde se usa
Variable de usuario	@x	consultas normales
Variable local	DECLARE x INT	procedimientos

✓ Ejercicio 1

Crea un script que asigne a una variable el valor 10 y a otra el valor 20. Muestra el resultado de la suma de ambas variables.

💡 Solución

```
-- Asignar valores
SET @var1 = 10;
SET @var2 = 20;

-- Mostrar suma
SELECT @var1 AS variable1, @var2 AS variable2, @var1 + @var2 AS suma;
```

✓ Ejercicio 2

Crea un script que obtenga el número total de jugadores de la tabla jugadores y lo almacene en una variable.

Solución

```
-- Se obtiene el total de jugadores
SELECT COUNT(*) INTO @numJugadores FROM jugadores;

-- Se muestra el resultado
SELECT @numJugadores AS TotalJugadores;
```

Ejercicio 3

Crea un script que obtenga el número de filas de cada tabla de la base de datos nba y visualice el resultado en una tabla con dos columnas: nombre de la tabla y número de filas.

Solución

```
-- Se obtiene el número de filas de cada tabla.
SELECT COUNT(*) INTO @numEquipos FROM equipos;
SELECT COUNT(*) INTO @numJugadores FROM jugadores;
SELECT COUNT(*) INTO @numEstadisticas FROM estadisticas;
SELECT COUNT(*) INTO @numPartidos FROM partidos;

-- Se muestra el resultado
SELECT 'equipos' AS NombreTabla, @numEquipos AS NumFilas
UNION ALL SELECT 'jugadores', @numJugadores
UNION ALL SELECT 'estadisticas', @numEstadisticas
UNION ALL SELECT 'partidos', @numPartidos;
```

Procedimientos

Resumen procedimientos

Sintáxis

```
DELIMITER //
```

```
CREATE PROCEDURE nombre()  
  BEGIN  
    instrucciones;  
  END //
```

```
DELIMITER ;
```

Llamada

```
CALL nombre()
```

Bloques (agrupan instrucciones)

```
BEGIN  
  ...  
END
```

Eliminar procedimientos

```
DROP PROCEDURE IF EXISTS datos_cliente;
```

Consultar procedimientos almacenados en la base de datos

```
SHOW PROCEDURE STATUS;
```

Variables locales al procedimiento

- Tipos: *Los mismos que se emplean en el DDL*
- Declaración `DECLARE variable tipo`
- Asignación de valores `SET variable = valor`
- Valor por defecto `DEFAULT`

Parámetros de entrada **IN**. Si no indicamos nada, los parámetros por defecto son de entrada

```
CREATE PROCEDURE entrada(p1 INT) ...  
-- Se puede pasar un valor  
CALL entrada(5);  
-- O una variable de usuario  
CALL entrada(@valor);
```

Parámetros de salida **OUT**. Devuelven un valor al finalizar el procedimiento

```
-- Para recibir el valor de salida, se debe usar siempre una variable de usuario (@)
CREATE PROCEDURE salida(p1 INT, OUT p2 INT) ...;
CALL salida(5, @resultado);
```

Parámetros de entrada/salida **INOUT**.

- Se lee como dato de entrada.
- El procedimiento puede modificar ese valor.
- El valor modificado se devuelve al terminar.

```
-- Para recibir el valor se debe usar siempre una variable de usuario (@)
CREATE PROCEDURE contar(INOUT cuenta INT, incremento INT) ...;
CALL contar(@cantidad, 5);
```

Estructuras de control

```
IF condicion THEN sentencia
  [ELSEIF condicion THEN sentencia] ...
  [ELSE sentencia]
END IF
```

```
CASE condicionQueTomaValor
  WHEN valor THEN sentencia
  [WHEN valor THEN sentencia] ...
  [ELSE sentencia]
END CASE
```

```
CASE
  WHEN condicionEvaluada THEN sentencia
  [WHEN condicionEvaluada THEN sentencia] ...
  [ELSE sentencia]
END CASE
```

```
WHILE condicion DO
  sentencias
END WHILE
```

✓ Ejercicio 4

Modifica el procedimiento datos_jugador para que reciba como entrada el código de un jugador.

```
DELIMITER //
DROP PROCEDURE IF EXISTS datos_jugador //
CREATE PROCEDURE datos_jugador()
BEGIN
    DECLARE num_temporadas INT;
    DECLARE total_puntos DECIMAL(10,2);

    SELECT COUNT(es.temporada), SUM(es.Puntos_por_partido)
    INTO num_temporadas, total_puntos
    FROM estadisticas es
    WHERE es.jugador = 106;

    SELECT j.codigo, j.Nombre, num_temporadas, total_puntos
    FROM jugadores j
    WHERE j.codigo = 106;
END //
DELIMITER ;
```

💡 Solución

```

DELIMITER //
DROP PROCEDURE IF EXISTS datos_jugador //
CREATE PROCEDURE datos_jugador(cod_jugador INT)
BEGIN
    DECLARE num_temporadas INT;
    DECLARE total_puntos DECIMAL(10,2);

    SELECT COUNT(es.temporada), SUM(es.Puntos_por_partido)
    INTO num_temporadas, total_puntos
    FROM estadisticas es
    WHERE es.jugador = cod_jugador;

    SELECT j.codigo, j.Nombre, num_temporadas, total_puntos
    FROM jugadores j
    WHERE j.codigo = cod_jugador;
END //
DELIMITER ;

--- Llamada
CALL datos_jugador(106);

```

✓ Ejercicio 5

Crea un procedimiento denominado `media_puntos_equipo`, que reciba como parámetro el nombre de un equipo y devuelva en una variable la media de puntos por partido de los jugadores de ese equipo.

💡 Solución

```

DROP PROCEDURE IF EXISTS media_puntos_equipo;
DELIMITER //
CREATE PROCEDURE media_puntos_equipo(IN equipo VARCHAR(20), OUT media DECIMAL(10,2))
BEGIN
    --- Se calcula la media de puntos del equipo
    SELECT AVG(e.Puntos_por_partido) INTO media
    FROM estadisticas e, jugadores j
    WHERE e.jugador = j.codigo AND j.Nombre_equipo = equipo;

    --- Si el equipo no existe o no existe ningún jugador con estadísticas, la función AVG dev
    --- CAPTURA DEL ERROR
    IF media IS NULL THEN
        --- Si media es NULL, lanzamos un error (SIGNAL) de tipo usuario (45000) con un texto (SE
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Equipo no encontrado o sin jugadores con esta
    END IF;
END //
DELIMITER ;

--- Llamada
SET @media := 0;
CALL media_puntos_equipo('Lakers', @media);
SELECT 'Lakers', @media AS MediaPuntos;

```

PROBAR A PARTIR DE AQUI!!!!

Ejercicio 6

Crea un procedimiento que calcule una predicción de puntos por partido de un jugador tras una mejora de rendimiento.

El procedimiento se denomina prediccion_puntos y recibe como parámetro la media de Puntos_por_partido de un jugador, devolviendo en el mismo parámetro la predicción de puntos tras aplicar un incremento del 10% (se devuelve la media de Puntos_por_partido incrementada en un 10%).

Solución

```

DROP PROCEDURE IF EXISTS proyeccion_puntos;
DELIMITER //

CREATE PROCEDURE proyeccion_puntos(INOUT puntos DECIMAL(10,2))
BEGIN
    -- Aplicamos el incremento del 10%
    SET puntos := puntos * 1.10;

    -- CAPTURA DE ERROR
    -- Si el valor es NULL, lanzamos error
    IF puntos IS NULL THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'El valor de puntos es NULL';
    END IF;

END //

DELIMITER ;

-- Llamada
SET @puntos := 20;
CALL proyeccion_puntos(@puntos);
SELECT @puntos AS PuntosProyectados;

```

Ejercicio 7

Calcular el bonus de un jugador

Crea un procedimiento denominado `bonus_jugador`, que devuelva el bonus de un jugador dado su identificador de jugador (*codigo*) y una temporada (*temporada*).

El bonus se calcula como un porcentaje de sus `Puntos_por_partido` según la siguiente tabla:

- Si los puntos por partido son menores que 10, el bonus es del 5%.
- Si los puntos por partido están entre 10 y 20, el bonus es del 10%.
- Si los puntos por partido son mayores que 30, el bonus es del 20%.

El procedimiento debe mostrar por pantalla el nombre del jugador, la temporada, sus puntos por partido y el bonus que ha obtenido. No se realiza ningún cambio en la base de datos.

Si el código del jugador no existe o no tiene estadísticas en la temporada indicada, se mostrará el mensaje *Jugador no encontrado o sin estadísticas*.

Solución

```
DROP PROCEDURE IF EXISTS bonus_jugador;
DELIMITER //

CREATE PROCEDURE bonus_jugador(IN cod_jugador INT, IN temp VARCHAR(5))
BEGIN
    DECLARE nombre_jugador VARCHAR(30);
    DECLARE puntos DECIMAL(10,2);
    DECLARE bonus DECIMAL(10,2) DEFAULT 0;

    -- Obtener nombre del jugador y sus puntos por partido en la temporada indicada
    SELECT j.Nombre, e.Puntos_por_partido INTO nombre_jugador, puntos
    FROM jugadores j, estadisticas e
    WHERE j.codigo = e.jugador
        AND j.codigo = cod_jugador
        AND e.temporada = temp;

    -- Captura del error
    IF puntos IS NULL THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Jugador no encontrado o sin estadísticas'
    END IF;

    -- Calcular bonus según los puntos por partido
    IF puntos < 10 THEN
        SET bonus := 0.05;
    ELSEIF puntos >= 10 AND puntos <= 20 THEN
        SET bonus := 0.1;
    ELSEIF puntos > 30 THEN
        SET bonus := 0.2;
    END IF;

    -- Mostrar resultado
    SELECT nombre_jugador AS Nombre, temp AS temporada, puntos AS Puntos_por_partido, bonus AS
END //
DELIMITER ;

-- Llamada
CALL bonus_jugador(106, '99/00');
CALL bonus_jugador(2, '00/01');
```

Ejercicio 8

Actualizar el peso de un jugador

Actualizar peso de un jugador

Crea un procedimiento que actualice el **Peso** de un jugador dado su identificador (*codigo*) y un porcentaje de incremento.

El procedimiento debe realizar las siguientes acciones:

- Si el porcentaje de incremento es negativo, mostrar un mensaje de error y no realizar ninguna actualización.
- Si el nuevo peso es superior a 150, mostrar un mensaje de advertencia indicando que el peso es muy alto.
- Si el nuevo peso es superior a 150, Actualizar el **Peso** del jugador en la tabla **jugadores**.
- Mostrar por pantalla el nombre del jugador, su nuevo peso y el nombre de su equipo.

Si el jugador no existe, se mostrará el mensaje *Jugador no encontrado*.

Solución

Solución **SQL**

Ejercicio 9

Calcular el total de una comanda

Crea un procedimiento que calcule el total de una comanda dado su número (COM_NUM).

El procedimiento debe realizar las siguientes acciones:

- Sumar el importe (IMPORT) de todos los detalles de la comanda (tabla DETALL).
- Actualizar el campo TOTAL de la tabla COMANDA con el valor calculado.
- Mostrar por pantalla el número de comanda y su total.

Solución

Solución **SQL**

✓ Ejercicio 10

Mostrar los empleados de un departamento

Crea un procedimiento que muestre por pantalla los empleados de un departamento dado su nombre (DNOM). El procedimiento debe mostrar el nombre, oficio y salario de cada empleado.

💡 Solución

Solución `SQL`

✓ Ejercicio 11

Contar productos por descripción

Crea un procedimiento que cuente la cantidad de productos vendidos dado una descripción. El procedimiento debe controlar si la descripción introducida es nula o vacía.

💡 Solución

Solución `SQL`

Funciones

Funciones integradas en el SGBD



Funciones integradas

Funciones matemáticas

Función	Descripción
ABS(x)	Valor absoluto de x.
FLOOR(x)	Parte entera inferior de x.
MOD(x,y)	Resto de la división x / y.
POW(x,y)	x elevado a y.
SQRT(x)	Raíz cuadrada de x.
RAND()	Número aleatorio entre 0 y 1.
ROUND(x,d)	Redondea x con d decimales.
ROUND(x)	Redondea x al entero más cercano.
SIGN(x)	1 si x>0, -1 si x<0, 0 si x=0.

Funciones de cadenas

Función	Descripción
CONCAT(c1,c2,...)	Concatena varias cadenas.
UPPER(c)	Convierte la cadena a mayúsculas (UCASE).
LOWER(c)	Convierte la cadena a minúsculas (LCASE).
LEFT(c,k)	Devuelve los k primeros caracteres.
RIGHT(c,k)	Devuelve los k últimos caracteres.
SUBSTRING(c,k,l)	Subcadena desde posición k con longitud l.
SUBSTRING_INDEX(c,pt,k)	Parte de la cadena según aparición del patrón pt.
INSTR(c,c2)	Posición donde aparece c2 en c (0 si no aparece).
LENGTH(c)	Número de caracteres de la cadena.
TRIM(c)	Elimina espacios al inicio y final.
REPEAT(c,k)	Repite la cadena k veces.
REPLACE(c,cb,cr)	Sustituye cb por cr en la cadena.
REVERSE(c)	Invierte la cadena.
STRCMP(c1,c2)	Compara cadenas (-1,0,1).

Funciones de fechas

Función	Descripción
CURDATE()	Fecha actual.
CURTIME()	Hora actual.
NOW()	Fecha y hora actual.
ADDDATE(f,d)	Suma d días a la fecha f.
ADDTIME(t,s)	Suma s segundos al tiempo t.
DATEDIFF(f1,f2)	Días entre f1 y f2.
TIMEDIFF(t1,t2)	Diferencia entre tiempos t1 y t2.
DAY(f)	Día del mes de la fecha f.
MONTH(f)	Mes de la fecha f.
YEAR(f)	Año de la fecha f.
DAYOFWEEK(f)	Día de la semana (1=domingo).
DAYNAME(f)	Nombre del día de la semana.
HOUR(t)	Hora del tiempo t.
MINUTE(t)	Minutos del tiempo t.
SECOND(t)	Segundos del tiempo t.
SEC_TO_TIME(s)	Convierte segundos a tiempo.
TIME_TO_SEC(t)	Convierte tiempo a segundos.
MAKETIME(h,m,s)	Crea un tiempo con h,m,s.
STR_TO_DATE(cf,ff)	Convierte texto a fecha.
DATE_FORMAT(f,ff)	Formatea una fecha como texto.

Funciones para mostrar información

Función	Descripción
VERSION()	Versión de MySQL
DATABASE()	Base de datos
CURRENT_USER()	Usuario
CONVERT(parm, tp)	Convierte parm al tipo tp
ISNULL(var)	1 si var es NULL, 0 en caso contrario
IFNULL(parm, msg)	Si parm es NULL retorna msg si no retorna parm

✓ Ejercicio 12

Crea un procedimiento que devuelva el promedio del salario de los empleados que trabajan en un departamento redondeado a dos decimales (ROUND). El procedimiento recibe como parámetro el número de departamento

💡 Solución

Solución **SQL**

✓ Ejercicio 13

Crea un procedimiento que devuelva el factorial de un número entero. El procedimiento recibe como parámetro el número entero y el resultado se devuelve en un parámetro de salida. Si el número es negativo se devolverá -1.

💡 Solución

Solución **SQL**

✓ Ejercicio 14

Crea un procedimiento que dado el código de un cliente muestre un código compuesto por las tres primeras letras del nombre (NOM) un guión y el código de cliente (CLIENT_COD). En el caso del cliente 104 y nombre EVERY MOUNTAIN, el código generado sería EVE-104. Funciones CONCAT y LEFT

💡 Solución

Solución **SQL**

✓ Ejercicio 15

Crea un procedimiento, que reciba como argumento una fecha en formato yyyy-mm-dd y devuelva en otro parámetro la fecha en formato 'día de la semana, día de mes, mes, año con cuatro dígitos (ejemplo: lunes, 12 de enero de 2025). Función DATE_FORMAT

💡 Solución

Solución **SQL**

✓ Ejercicio 16

Crea un procedimiento que dado el código de un producto (PROD_NUM) devuelva el precio de venta máximo, mínimo y medio (tabla DETALL). El procedimiento recibe como parámetro el código de producto y devuelve los valores en tres parámetros de salida. Funciones MAX , MIN y AVG .

💡 Solución

Solución **SQL**

Funciones propias



Resumen funciones

Sintaxis

```
CREATE FUNCTION nombre (parámetros) RETURNS tipo
BEGIN
    bloque con RETURN
END;
```

Llamada

```
SELECT nombre(parámetros);

-- Uso
SELECT precioConIVA(1234);

-- Consulta de una tabla con un campo calculado
SELECT COM_NUM, TOTAL AS Neto, precioConIVA(TOTAL) AS 'Total con Iva' FROM COMANDA;
```

Eliminar funciones

Ejercicio 1. Crea una función que devuelva el nombre del departamento dado un código de departamento. Llama a la función desde un SELECT que muestre el nombre del empleado y el nombre del departamento, ordenado por departamento.

Ejercicio 2. Verificar si un producto existe. Crea una función que verifique si un producto existe en la base de datos, dado su número de producto (PROD_NUM). La función debe devolver un valor booleano (1 si existe, 0 si no existe).

Ejercicio 3. Crea una función que cuente el número de pedidos realizados por un cliente, dado su código de cliente (CLIENT_COD). La función debe devolver un valor numérico que represente el número de pedidos del cliente. Ejecuta la función desde un select que muestre el nombre del cliente, el número de pedidos y el total de ventas.

Ejercicio 4. Calcular la antigüedad de un empleado. Crea una función que calcule la antigüedad de un empleado en años, dado su número de empleado (EMP_NO). La función debe devolver un valor numérico que represente la cantidad de años transcurridos desde la fecha de alta (DATA_ALTA) del empleado. Utiliza YEAR() y CURDATE(). Ejecuta la función desde un select que muestre el nombre del empleado, la fecha de alta y la antigüedad en años. Utiliza